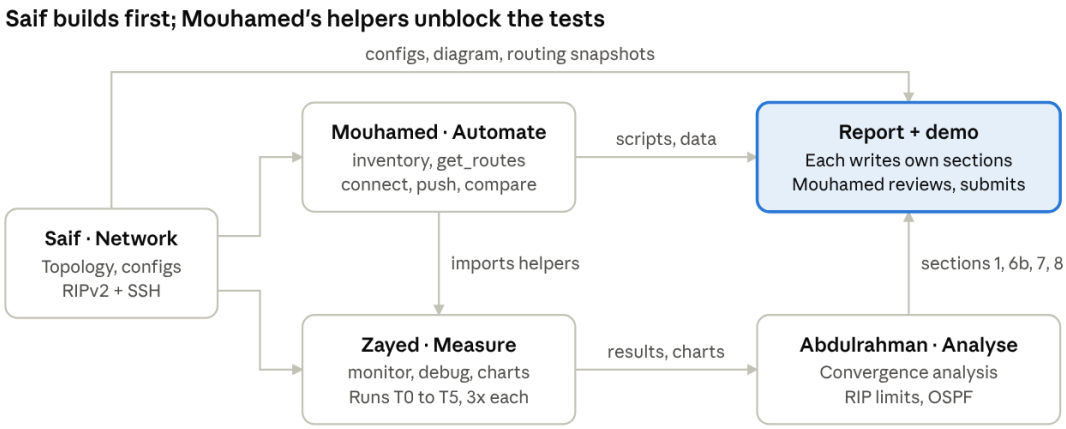
CCEN 356 Project 1: Team Plan & Technical Guide (v3)

Oct 20, 2026 · @HamSudo

Overview

The project now splits four ways: Saif builds the RIP network, Mouhamed writes the core Python automation, Zayed measures and visualises convergence, and Abdulrahman analyses the results and leads the demo. Each member writes their own report sections; Mouhamed reviews the full report before submitting.

Order of work: Saif's network first, then Mouhamed's `inventory.py` and `get_routes.py` (the measurement scripts build on them), then the tests, then the analysis. Scripts and theory can be drafted before the lab is ready.



team workflow · 4 parts feeding one report and demo

Saif's network and Mouhamed's two helpers are the only blockers; Abdulrahman can draft the theory sections now.

Changes from v1: a fourth member joined; Mouhamed and Abdulrahman swapped roles; the debug and convergence scripts, test execution and a new `visualize.py` move to Zayed; Mouhamed does the final report review instead of Abdulrahman assembling it.

Changes from v2: fixes from checking the plan against the course brief. A new `parse_rip_debug.py` (the brief asks for parsing logs as well as routing tables); `push_config.py` now triggers T5 in the demo; paste-in configs and running-config exports kept in separate folders; timestamps and router clocks set before testing; T1 to T5 expectations, T2, T4 and the ping method corrected; the parser records routes in hold-down.

Responsibilities

All four present in the demo.

Student	Owns	Delivers	Report sections
Saif	Network design and Cisco configuration	Topology diagram, IP plan, router and switch configs (RIPv2, no auto-summary, SSH), verified network, .txt config exports	2 Network design, 3 RIP configuration, 4 Verification and normal-state routing tables
Mouhamed	Core Python automation	Final review and submission of the full report; repo and README; `inventory.py`, `connect_test.py`, `push_config.py`, `get_routes.py` (JSON/CSV snapshots), `compare_routes.py`, `parse_rip_debug.py`	5 Python automation; appendix script listings
Zayed	Measurement, testing and visualisation	`capture_rip_debug.py`, `monitor_convergence.py`, `visualize.py`; runs T0 to T5; results table, debug logs, before/after snapshots, charts	6a Test procedure and results
Abdulrahman	Analysis and demo	Convergence analysis, RIP limitations, OSPF comparison, demo run-sheet and rehearsal	1 Introduction, 6b Convergence analysis, 7 Limitations of RIP, 8 Conclusion

Technical approach

1. **Build.** Three Cisco 2901 routers in a triangle (Library, Teachers' room, Laboratory), so every room has two paths to every other room. Each router connects to a 2960 switch with PCs, laptops and a printer.
2. **Address.** One /24 per room (192.168.10.0, 20.0, 30.0) and one /30 per router-to-router link (10.0.12.0, 10.0.13.0, 10.0.23.0).
3. **Configure RIP.** `router rip` > `version 2` > `no auto-summary` > `network` for each connected network.
4. **Verify.** `show ip route` shows R routes to the other rooms; PCs in different rooms ping each other.
5. **Enable SSH** on the routers so Python can log in.
6. **Automate.** netmiko logs in, pushes config and saves routing tables as JSON and CSV.
7. **Break and measure.** Shut a link, time how long RIP takes to recover, compare tables before and after.
8. **Visualise.** Chart convergence times and route changes from the collected data.
9. **Write up and demo.**

Where it runs. Packet Tracer covers steps 1 to 5 and 7. netmiko on a real laptop cannot reach a Packet Tracer network, so steps 6 to 8 run on the physical 2901s in the lab. The brief names Cisco 2901s, so the lab is the target. Only if it is unavailable, rebuild the design in GNS3 / EVE-NG with a Dynamips image that has serial ports (c7200 or c3725). IOSv has no serial interfaces, and shutting one end of a virtual link leaves the far end up, so T1 to T3 would behave like T5; say so in the report if that happens.

Saif: network design and Cisco configuration

Unchanged from v1. Design and configs are written in full in Part 1 (Saif): Network Design & RIP Configuration; what remains is the build, verification and export. Its handoff section still uses the v1 roles: SSH details now go to Mouhamed and the working network to Zayed.

Router	Room	LAN (G0/0)	Links
LIB-R1	Library	192.168.10.1/24	S0/0/0 to TCH-R2 (10.0.12.0/30, DCE); G0/1 to LAB-R3 (10.0.13.0/30)
TCH-R2	Teachers' room	192.168.20.1/24	S0/0/0 to LIB-R1; S0/0/1 to LAB-R3 (10.0.23.0/30, DCE)
LAB-R3	Laboratory	192.168.30.1/24	G0/1 to LIB-R1; S0/0/0 to TCH-R2

1. Add an HWIC-2T serial module to each 2901.
2. Cable the triangle: two serial links (DCE end sets `clock rate 64000`) and one Ethernet cross-over link.
3. Paste the router configs, then the switch configs.
4. Before testing, add `service timestamps debug datetime msec` and `service timestamps log datetime msec` to each router and set its clock (`clock set`), so debug and log lines carry usable times.
5. Verify with `show ip interface brief`, `show ip protocols`, `show ip route`, `debug ip rip` and cross-room pings.
6. Export each `show running-config` to `configs/exports/` (the paste-in scripts stay in `configs/source/`) and save `school-rip.pkt`.
7. Hand SSH details to Mouhamed and the working network to Zayed.

Mouhamed: core Python automation

Mouhamed writes the scripts that log into the routers over SSH, push config and save routing tables, plus the shared helpers Zayed's scripts import. Setup: Python 3, `pip install -r requirements.txt`, run from AUTO-PC (192.168.30.20) or any laptop that can reach the routers.

Connection details (from Saif's build; enable secret `class123`). `inventory.py` reads them from the `RIP_USER`, `RIP_PASS` and `RIP_SECRET` environment variables and falls back to these lab defaults, so real passwords never go into the repo.

Router	Host	Username	Password	device_type
LIB-R1	192.168.10.1	admin	Admin@123	cisco_ios

Router	Host	Username	Password	device_type
TCH-R2	192.168.20.1	admin	Admin@123	cisco_ios
LAB-R3	192.168.30.1	admin	Admin@123	cisco_ios

Scripts

Script	What it does	netmiko calls
`inventory.py`	Router list and a `connect(name)` helper; imported by every other script	`ConnectHandler`
`connect_test.py`	Logs into each router and prints its hostname; proves SSH works	`find_prompt()`
`push_config.py`	Applies commands from a text file (e.g. the RIP block, or the T5 `passive-interface` change), then saves	`send_config_set()`, `save_config()`
`get_routes.py`	Runs `show ip route rip`, parses each route, saves a timestamped JSON snapshot and appends to `routes.csv`	`send_command()`
`compare_routes.py`	Compares two JSON snapshots; prints added, removed and changed routes (next hop, metric or status). Matches on network + next hop, since equal-cost paths give one network two entries	none
`parse_rip_debug.py`	Reads the captures in `data/debug_logs/`; pulls out each sent and received update with its routes, interface up/down lines and routing-table changes (`RT: add` / `delete`); writes a CSV and measures convergence from the interface change (or the last update heard, for T5) to the new route's install. Covers the brief's "parsing logs"	none

Shared interface for Zayed. Ship these two first, since the measurement scripts depend on them:

- `inventory.connect(name)` returns a connected netmiko session for `LIB-R1`, `TCH-R2` or `LAB-R3`.
- `get_routes.parse_routes(text)` turns `show ip route rip` output into a list of `{network, metric, next_hop, interface, status}`, where status is `up` or `possibly_down`.

Parser notes. Lines look like `R 192.168.20.0/24 [120/1] via 10.0.12.2, 00:00:12, Serial0/0/0`. The age (`00:00:12`) is in real output but not in Saif's expected tables, so make it optional. With two equal-cost paths, the second path prints on its own line with no network in front; attach it to the last network seen. A route in hold-down prints over two lines (`R 192.168.30.0/24 is possibly down`, then `routing via 10.0.13.2, GigabitEthernet0/1`); record it with status `possibly_down` and metric 16, not skipped. Skip the code legend, `Gateway of last resort` and `is variably`

`subnetted` header lines. The parser can be tested against the expected tables in Saif's doc before the lab is ready.

Snapshot format

```
{
  "time": "2026-10-20T10:15:02",
  "label": "T1-before",
  "routers": {
    "LIB-R1": [
      {"network": "192.168.20.0/24", "metric": 1, "next_hop": "10.0.12.2",
"interface": "Serial0/0/0", "status": "up"},
      {"network": "192.168.30.0/24", "metric": 1, "next_hop": "10.0.13.2",
"interface": "GigabitEthernet0/1", "status": "up"}
    ]
  }
}
```

Zayed: measurement, testing and visualisation

Zayed writes the measurement tools, runs every link-failure test with them, and turns the results into charts for the report.

Scripts (import `inventory` and `get_routes` from Mouhamed)

Script	What it does	netmiko calls
`capture_rip_debug.py`	Turns on `terminal monitor`, `debug ip rip` and `debug ip routing`, records output to `data/debug_logs/` for 60 s (about 300 s for T5, which takes 240 s or more), then `undebug all`	`send_command_timing()`, `read_channel()`
`monitor_convergence.py`	Polls one router every 1 s over one open session and logs the exact time a chosen route changes next hop, metric or status (hold-down); reconnects and logs the gap if the session drops (it will in T2). Times are to the nearest second	`send_command()` in a loop
`visualize.py`	Reads the results and monitor logs; plots average convergence per test (with min–max) and a route-change timeline for one test	none (matplotlib)

Handoff from Mouhamed. The tests and these scripts depend on the following:

- Push `configs/push/timestamps.txt` to all three routers before the first test (`push_config.py configs/push/timestamps.txt all`). It adds the msec timestamps and a 64 KB log buffer.

- Capture `debug ip routing` as well as `debug ip rip`. Convergence ends at the `RT: add` line where the new route is installed; without it `parse_rip_debug.py` cannot measure anything.
- Record about 300 s for T5 (it converges after 240 s or more); 60 s is enough for T1 to T4.
- T2 cuts the SSH path to LIB-R1, so a live capture drops. Run `clear logging` before each run and read the capture back with `show logging` afterwards; the buffer holds it.
- **Suggestion:** since the log buffer is on anyway, `capture_rip_debug.py` could skip the live `terminal monitor` capture altogether: `clear logging`, turn on the debugs, run the test, then save `show logging` to the file. One method for every test, and nothing is lost when an SSH session drops. Zayed's call.
- Name captures `<test>-run<n>_<router>.txt`, e.g. `T1-run1_LIB-R1.txt`, in `data/debug_logs/`.
- Measure each run with `parse_rip_debug.py <log> --network <net> --metric <m>`: T1 on LIB-R1, 192.168.20.0/24 metric 2; T2 on LIB-R1, 192.168.30.0/24 metric 2; T3 on TCH-R2, 192.168.30.0/24 metric 2; T4 on LIB-R1 (T1 link restored), 192.168.20.0/24 metric 1; T5 on LIB-R1, 192.168.30.0/24 metric 2 plus `--silent-peer 10.0.13.2`.
- `monitor_convergence.py` uses `get_routes.parse_routes()`; a route in hold-down has status `possibly_down` and metric 16, so watch that field for T5.

Test plan

Test	How to cause it	What should happen
T0 Baseline	All links up; run `get_routes.py`	Every remote LAN at metric 1
T1 Library–Teachers serial down	LIB-R1: `interface s0/0/0` > `shutdown`	LIB-R1 reaches 192.168.20.0/24 via LAB-R3, metric 2
T2 Library–Lab Ethernet down	LIB-R1: `interface g0/1` > `shutdown`, from the console: AUTO-PC's SSH session to LIB-R1 runs over this link and drops	LIB-R1 reaches 192.168.30.0/24 via TCH-R2, metric 2
T3 Teachers–Lab serial down	TCH-R2: `interface s0/0/1` > `shutdown`	TCH-R2 reaches 192.168.30.0/24 via LIB-R1, metric 2
T4 Recovery	`no shutdown` on the T1 link, the same link every run	Original 1-hop route returns
T5 Silent failure	LAB-R3: `push_config.py` applies `router rip` > `passive-interface g0/1` (stops updates, link stays up)	LIB-R1 marks 192.168.30.0/24 possibly down when the 180 s invalid timer expires and flushes it at 240 s; the metric-2 route via TCH-R2 arrives with TCH-R2's next update: expect about 240 to 270 s

T1 to T3 react at once because the router sees its own interface go down, but the new path only arrives with a neighbour's next update, so expect anywhere from about 0 to 30 s and a spread between runs. T5 is the key result: nothing goes down, so RIP waits for its timers. Traffic keeps flowing while the stale

route is in hold-down, so expect little or no ping loss (a short gap is possible between the flush and TCH-R2's next update), and expect asymmetric routing afterwards (LAB-R3 still hears LIB-R1 and keeps its direct route). Snapshot all three routers before and after every test.

Measuring convergence (at least two methods, 3 runs per test, report the average)

- **Timestamped debug:** push `configs/push/timestamps.txt`, then capture `debug ip rip` and `debug ip routing` on the affected router. Router clocks are not synchronised, so both times come from that router: from the `%LINK` / `%LINEPROTO` line (for T5, the last update heard from LAB-R3, since the invalid timer counts from it) to the `RT: add` line where the new route is installed. The first update carrying the new route is not the end point: the backup path is advertised all along and ignored until the old route is gone. `parse_rip_debug.py` measures this.
- **Python monitor:** `monitor_convergence.py` on the affected router.
- **Ping loss:** continuous ping between two PCs with a 1 s timeout (`ping -t -w 1000` on Windows; the default 4 s timeout makes each lost reply worth about 4 s); lost replies × 1 s ≈ outage. Expect little or none for T5.

Results table

Test	Run 1 (s)	Run 2 (s)	Run 3 (s)	Average (s)	Pings lost (avg)
T1					
T2					
T3					
T4					
T5					

Abdulrahman: analysis and demo

Abdulrahman explains the results, covers RIP's limitations, and leads the demo. The theory sections can be drafted now, before any results exist.

Convergence analysis (6b)

- Why T1 to T3 recover fast: the interface goes down and the route is removed at once, but the new path only arrives with the neighbour's next update (0 to 30 s), which explains the spread between runs.
- Why T5 is slow: no interface event, so the stale route survives until the invalid timer (180 s), sits in hold-down, and is replaced only after the flush timer (240 s) removes it and TCH-R2's next update arrives. Hold-down alone would last until 360 s; the flush cuts it short.
- How each measured average compares with the expected timer values; why the methods disagree.
- The asymmetric routing left after T5.

Limitations of RIP (7)

- Maximum 15 hops; 16 means unreachable.
- Hop count ignores bandwidth: our Ethernet link and 64 kbps serial links count the same.
- Slow convergence: update 30 s, invalid 180 s, hold-down 180 s, flush 240 s.
- Full table sent every 30 s, even with no change.
- Routing loops and count-to-infinity, softened only by split horizon, route poisoning and hold-down.
- No areas or hierarchy; close with a one-paragraph comparison with OSPF.

Demo. Owns the run-sheet below and the team rehearsal.

Demo run-sheet (about 10 minutes)

1. **Saif:** show the topology, `show ip route` on one router, ping across rooms.
2. **Saif:** `debug ip rip` for one update cycle.
3. **Mouhamed:** run `connect_test.py`, then `get_routes.py`; open the JSON snapshot.
4. **Zayed:** start `monitor_convergence.py`, shut the T1 link; the monitor shows the route change and the ping recovers.
5. **Mouhamed:** `compare_routes.py` on the before and after snapshots.
6. **Mouhamed:** `push_config.py` applies the T5 change (`passive-interface g0/1` on LAB-R3) from a file, to show configuration pushed from Python. T5 takes about 4 minutes, so it runs during the next steps.
7. **Zayed:** restore the link; show the convergence chart from `visualize.py`.
8. **Abdulrahman:** explain the T5 result while `get_routes.py` shows LIB-R1's route go possibly down and then switch to TCH-R2 (if it has not switched yet, show the possibly-down route and explain the remaining wait); summarise RIP's limitations. Remove the passive-interface afterwards.

Repo and report

One GitHub repo, `ccen356-rip-project`, holds everything the brief lists under Source Code Repository. Mouhamed sets it up; each student commits their own folder.

```
ccen356-rip-project/
  README.md           how to run the scripts (Mouhamed)
  requirements.txt    netmiko, matplotlib, pytest
  configs/source/     paste-in configs: LIB-R1.txt, TCH-R2.txt, LAB-R3.txt, SW-*.txt
(Saif)
  configs/exports/    show running-config exports from the built devices (Saif)
  topology/           school-rip.pkt, topology.png (Saif)
  scripts/            inventory.py, connect_test.py, push_config.py,
                      get_routes.py, compare_routes.py,
                      parse_rip_debug.py (Mouhamed)
                      capture_rip_debug.py, monitor_convergence.py,
                      visualize.py (Zayed)
  tests/              parser tests against Saif's expected tables (Mouhamed)
  data/               snapshots/*.json, routes.csv, debug_logs/*.txt,
                      monitor_logs/*.csv, charts/*.png (Zayed)
  docs/               final report PDF (Mouhamed)
```

Report outline (each owner writes their sections; Mouhamed reviews the full report and submits)

#	Section	Writer
1	Introduction and objectives	Abdulrahman
2	Network design: topology diagram, IP plan, cabling	Saif
3	RIP configuration details	Saif
4	Verification and routing table snapshots	Saif
5	Python automation scripts	Mouhamed
6a	Link-failure tests and results	Zayed
6b	Convergence analysis	Abdulrahman
7	Limitations of RIP	Abdulrahman
8	Conclusion	Abdulrahman
A	Appendix: full configs and script listings	Saif, Mouhamed, Zayed

Checklist

Saif

- ☒ Topology, IP plan and cabling designed
- ☒ Router and switch configs written
- ☐ Network built in Packet Tracer and in the lab
- ☐ Timestamp lines added and router clocks set before testing
- ☐ All verification checks pass; routing table screenshots saved
- ☐ Running configs exported to `configs/exports/` and committed
- ☐ Report sections 2 to 4 written

Mouhamed

- ☐ Repo created with the layout above
- ☐ `inventory.py` and `get_routes.parse_routes` shared with Zayed
- ☐ `connect_test.py` logs into all 3 routers
- ☐ `push_config.py` applies config from a file (RIP block, T5 change)
- ☐ `get_routes.py` saves JSON and CSV snapshots
- ☐ `compare_routes.py` shows differences between two snapshots
- ☐ `parse_rip_debug.py` extracts update and link-event times from the debug logs
- ☐ README and report section 5 written; full report reviewed and submitted

Zayed

- ☐ `capture_rip_debug.py` and `monitor_convergence.py` working (monitor reconnects after T2)
- ☐ Captures include `debug ip routing`; T5 recorded for about 300 s
- ☐ Tests T0 to T5 run, 3 times each
- ☐ Results table filled in with averages
- ☐ Before/after snapshots and debug logs saved for each test
- ☐ `visualize.py` charts saved to `data/charts/`
- ☐ Report section 6a written

Abdulrahman

- ☐ Sections 1, 7 and 8 drafted
- ☐ Convergence analysis (6b) written from the results
- ☐ Demo run-sheet finalised
- ☐ Demo rehearsed with the whole team